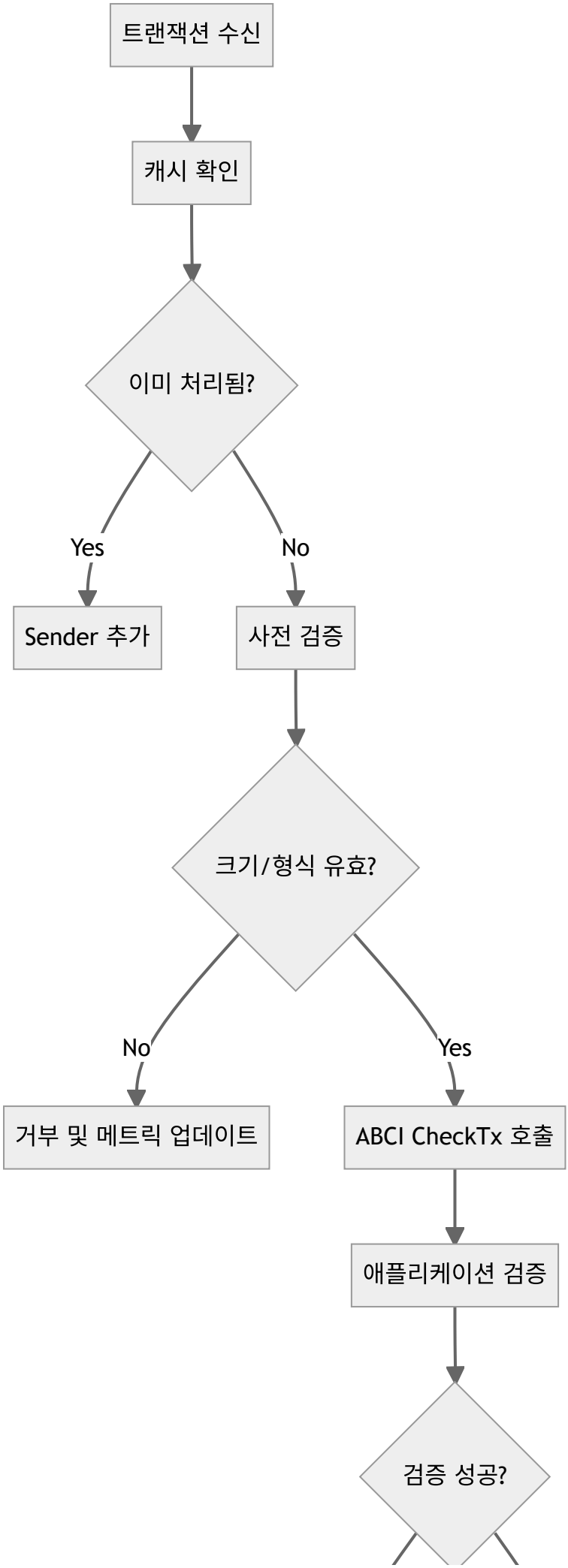
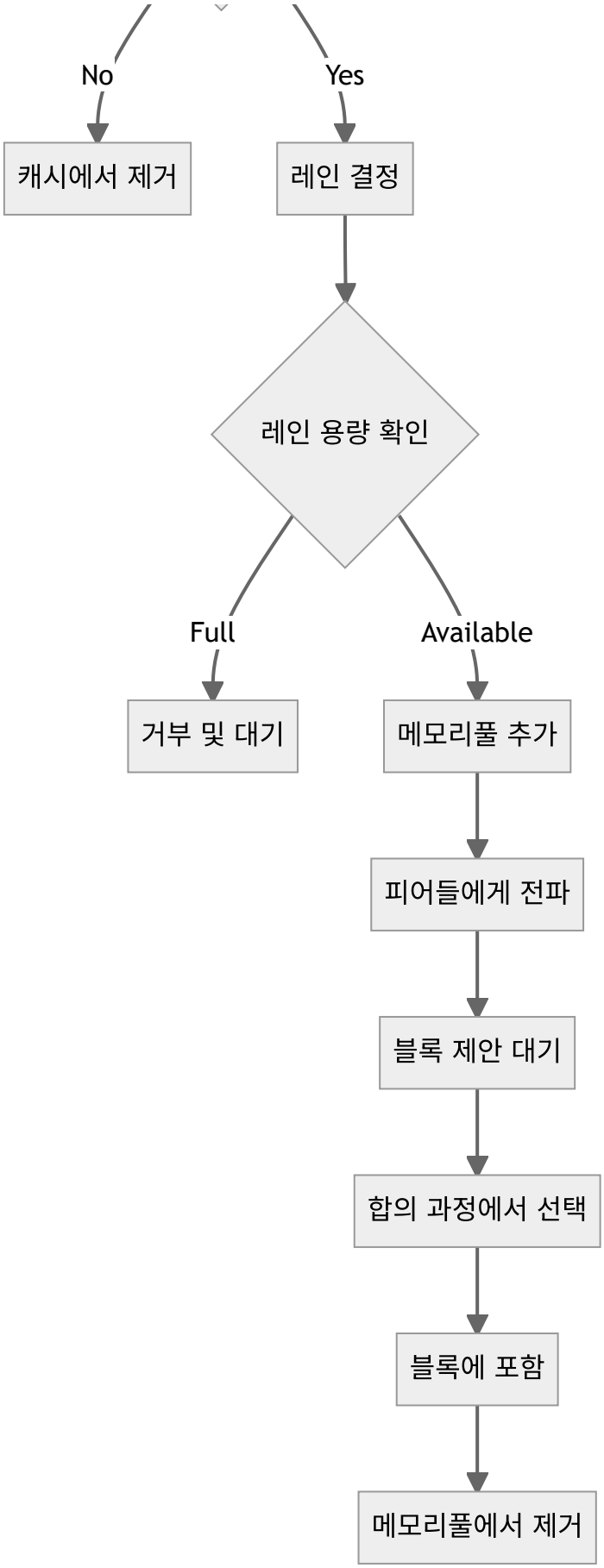
트랜잭션 처리 과정

개요

CometBFT 메모리풀에서의 트랜잭션 처리는 수신부터 블록 포함까지의 복잡한 과정을 거칩니다. 이 문서는 트랜잭션이 네트워크에서 수신되어 최종적으로 블록에 포함되기까지의 전체 생명주기를 상세히 설명합니다.

트랜잭션 생명주기 개요


트랜잭션 수신
캐시 확인
이미 처리됨?
Yes
No
Sender 추가
사전 검증
크기/형식 유효?
No
Yes
거부 및 메트릭 업데이트
ABCI CheckTx 호출
애플리케이션 검증
검증 성공?



1. 트랜잭션 수신 단계

P2P 네트워크를 통한 수신

트랜잭션은 주로 두 가지 경로로 메모리풀에 도달합니다:

1. **클라이언트 직접 제출**: RPC를 통해 직접 제출
2. **피어 전파**: 다른 노드로부터 P2P 네트워크를 통해 수신

```go
// Reactor에서 트랜잭션 메시지 처리
func (memR *Reactor) Receive(chID  byte, src p2p.Peer, msgBytes []byte) {
```

```go
switch chID {
case MempoolChannel:
// 트랜잭션 배치 메시지 처리
msg := &protomem.Txs{}
for _, tx := range msg.Txs {
// 각 트랜잭션을 메모리풀에 추가 시도
memR.mempool.CheckTx(tx, src.ID())
}
}
}
```

초기 검증

수신된 트랜잭션은 즉시 기본적인 검증을 거칩니다:

```go
func (mem *CListMempool) CheckTx(tx types.Tx, sender p2p.ID) (*abcicli.ReqRes, e
// 1. 메모리풀 용량 확인
if err := mem.isFull(len(tx)); err != nil {
mem.metrics.RejectedTxs.Add(1)
return nil, err
}
// 2. 트랜잭션 크기 확인
if len(tx) > mem.config.MaxTxBytes {
return nil, ErrTxTooLarge{Max: mem.config.MaxTxBytes, Actual: len(tx)}
```

```
}

// 3. 사전 검증 함수 실행 (설정된 경우)

if  mem.preCheck != nil {

if  err := mem.preCheck(tx);  err != nil {

return  nil, ErrPreCheck{Err: err}

}

}

}
```

2. 캐시 확인 및 중복 처리

LRU 캐시 동작

메모리풀은 LRU(Least Recently Used) 캐시를 사용하여 중복 트랜잭션을 효율적으로 필터링합니다:

```
func (mem *CListMempool) CheckTx(tx types.Tx, sender p2p.ID) (*abcicli.ReqRes, e

// 캐시에 추가 시도

if  added := mem.addToCache(tx); !added {

// 이미 캐시에 존재하는 트랜잭션

mem.metrics.AlreadyReceivedTxs.Add(1)

// 새로운 sender 정보 추가

if  err := mem.addSender(tx.Key(), sender);  err != nil {

mem.logger.Error("Could not add sender to tx", "tx", tx.Hash(), "sender", sender

}
```

```
return nil, ErrTxInCache
}
}
```

Sender 추적

각 트랜잭션은 어떤 피어로부터 수신되었는지 추적합니다:

```
type mempoolTx struct {
// ... 다른 필드들
senders sync.Map // 트랜잭션을 전송한 피어들의 ID 맵
}

func (memTx *mempoolTx) addSender(peerID p2p.ID) bool {
if len(peerID) == 0 {
return false
}
_, loaded := memTx.senders.LoadOrStore(peerID, struct{}{})
return loaded // true면 이미 존재했던 sender
}
```

3. ABCI 검증 과정

비동기 CheckTx 호출

트랜잭션은 ABCI 인터페이스를 통해 애플리케이션에서 검증됩니다:

```
func (mem *CListMempool) CheckTx(tx types.Tx, sender p2p.ID) (*abcicli.ReqRes, e

// 비동기 CheckTx 요청

reqRes, err := mem.proxyAppConn.CheckTxAsync(context.TODO(), &abci.CheckTxReques

Tx: tx,

Type: abci.CHECK_TX_TYPE_CHECK,

})

if  err != nil {

panic(fmt.Errorf("CheckTx request failed: %w", err))

}

// 응답 처리 콜백 설정

reqRes.SetCallback(mem.handleCheckTxResponse(tx, sender))

return  reqRes, nil

}
```

CheckTx 응답 처리

애플리케이션으로부터의 응답을 처리하는 과정:

```
func (mem *CListMempool) handleCheckTxResponse(tx types.Tx, sender p2p.ID) func(

return  func(r *abci.Response) error {

res := r.GetCheckTx()

// 1. 사후 검증 (PostCheck) 실행
```

```
var postCheckErr error
if mem.postCheck != nil {
postCheckErr = mem.postCheck(tx, res)
}
// 2. 검증 실패 시 처리
if res.Code != abci.CodeTypeOK || postCheckErr != nil {
mem.tryRemoveFromCache(tx)
mem.metrics.FailedTxs.Add(1)
return ErrInvalidTx{Code: res.Code, Data: res.Data, Log: res.Log}
}
// 3. 레인 결정
lane := mem.defaultLane
if res.LaneId != "" {
lane = LaneID(res.LaneId)
}
// 4. 레인 용량 확인
if err := mem.isLaneFull(len(tx), lane); err != nil {
mem.forceRemoveFromCache(tx)
return err
}
// 5. 메모리풀에 추가
mem.addTx(tx, res.GasWanted, sender, lane)
mem.notifyTxsAvailable()
```

```
return nil
}
}
```

4. 레인 관리 및 우선순위

레인 결정 로직

애플리케이션은 CheckTx 응답에서 트랜잭션이 속할 레인을 지정할 수 있습니다:

```
// ABCI CheckTx 응답에서 레인 정보 추출
lane := mem.defaultLane
if res.LaneId != "" {
if _, ok := mem.lanes[LaneID(res.LaneId)]; !ok {
panic(ErrLaneNotFound{laneID: LaneID(res.LaneId)})
}
lane = LaneID(res.LaneId)
}
```

레인별 용량 관리

각 레인은 전체 메모리풀 용량을 균등하게 분할하여 사용합니다:

```
func (mem *CListMempool) isLaneFull(txSize int, lane LaneID) error {
laneTxs, laneBytes := mem.LaneSizes(lane)
```

```
// 레인별 용량 계산
laneTxsCapacity := mem.config.Size / len(mem.sortedLanes)
laneBytesCapacity := mem.config.MaxTxsBytes / int64(len(mem.sortedLanes))
if  laneTxs > laneTxsCapacity || int64(txSize)+laneBytes > laneBytesCapacity {
return ErrLaneIsFull{
Lane: lane,
NumTxs: laneTxs,
MaxTxs: laneTxsCapacity,
Bytes: laneBytes,
MaxBytes: laneBytesCapacity,
}
}
return  nil
}
```

5. 메모리풀 추가 과정

트랜잭션 저장

검증된 트랜잭션은 메모리풀에 추가됩니다:

```
func (mem *CListMempool) addTx(tx types.Tx, gasWanted  int64, sender p2p.ID, lan
mem.txsMtx.Lock()
defer  mem.txsMtx.Unlock()
```

```go
// 1. 레인의 연결 리스트 가져오기
txs, ok := mem.lanes[lane]
if !ok {
panic(ErrLaneNotFound{laneID: lane})
}
// 2. 시퀀스 번호 증가
mem.addTxSeq++
mem.addTxLaneSeqs[lane] = mem.addTxSeq
// 3. mempoolTx 객체 생성
memTx := &mempoolTx{
tx: tx,
height: mem.height.Load(),
gasWanted: gasWanted,
lane: lane,
seq: mem.addTxSeq,
timestamp: time.Now().UTC(),
}
memTx.addSender(sender)
// 4. 연결 리스트에 추가
e := txs.PushBack(memTx)
// 5. 인덱스 업데이트
mem.txsMap[tx.Key()] = e
mem.txsBytes += int64(len(tx))
```

```
mem.numTxs++

mem.laneBytes[lane] += int64(len(tx))

// 6. 이터레이터에게 알림

close(mem.addTxCh)

mem.addTxCh = make(chan  struct{})

}
```

6. 트랜잭션 전파

피어들에게 브로드캐스트

새로 추가된 트랜잭션은 연결된 피어들에게 전파됩니다:

```
// Reactor의 브로드캐스트 루틴

func (memR *Reactor) broadcastTxRoutine(peer p2p.Peer) {

peerID := peer.ID()

var  next *clist.CElement

for {

if !memR.IsRunning() || !peer.IsRunning() {

return

}

// 다음 트랜잭션 대기

if  next == nil {

select {
```

```
case <-memR.mempool.TxsAvailable():
next = memR.mempool.TxsFront()
case <-peer.Quit():
return
case <-memR.Quit():
return
}
}
// 트랜잭션 전송
memTx := next.Value.(*mempoolTx)
if !memTx.IsSender(peerID) {
success := peer.Send(MempoolChannel, cdc.MustMarshal(&TxMessage{Tx: memTx.tx}))
if !success {
time.Sleep(PeerCatchupSleepIntervalMS * time.Millisecond)
continue
}
}
next = next.Next()
}
}
```

DOG 프로토콜 (선택적)

대역폭 최적화를 위한 DOG(Dissemination Optimization through Gossip) 프로토콜:

```go
type gossipRouter struct {
routes  map[p2p.ID]bool  // 각 피어별 활성화 상태
}


func (r *gossipRouter) shouldGossipTo(peerID p2p.ID) bool {
r.mtx.RLock()
defer  r.mtx.RUnlock()
return  r.routes[peerID]
}
```

7. 블록 제안 및 선택

트랜잭션 수집 (Reaping)

합의 엔진이 블록을 제안할 때 메모리풀에서 트랜잭션을 수집합니다:

```go
func (mem *CListMempool) ReapMaxBytesMaxGas(maxBytes, maxGas  int64) types.Txs {
mem.updateMtx.RLock()
defer  mem.updateMtx.RUnlock()
var  totalGas, runningSize  int64
txs := make([]types.Tx, 0, mem.Size())
// IWRR 이터레이터를 사용하여 우선순위 기반 선택
iter := NewNonBlockingIterator(mem)
for {
```

```
memTx := iter.Next()
if memTx == nil {
break
}
dataSize := types.ComputeProtoSizeForTxs([]types.Tx{memTx.Tx()})
// 크기 제한 확인
if maxBytes > -1 && runningSize+dataSize > maxBytes {
return txs[:len(txs)-1]
}
// 가스 제한 확인
newTotalGas := totalGas + memTx.GasWanted()
if maxGas > -1 && newTotalGas > maxGas {
return txs[:len(txs)-1]
}
txs = append(txs, memTx.Tx())
runningSize += dataSize
totalGas = newTotalGas
}
return txs
}
```

IWRR (Interleaved Weighted Round Robin) 알고리즘

트랜잭션 선택 시 레인별 우선순위를 고려한 공정한 선택을 보장합니다:

```
func (iter *NonBlockingIterator) Next() Entry {
numEmptyLanes := 0
lane := iter.sortedLanes[iter.laneIndex]
for {
// 빈 레인 건너뛰기
if iter.cursors[lane.id] == nil {
numEmptyLanes++
if numEmptyLanes >= len(iter.sortedLanes) {
return nil
}
lane = iter.advanceIndexes()
continue
}
// 현재 라운드에서 과소비된 레인 건너뛰기
if int(lane.priority) < iter.round {
numEmptyLanes = 0
lane = iter.advanceIndexes()
continue
}
break
}
```

```
// 다음 트랜잭션 반환
elem := iter.cursors[lane.id]
iter.cursors[lane.id] = iter.cursors[lane.id].Next()
iter.advanceIndexes()
return elem.Value.(*mempoolTx)
}
```

8. 블록 커밋 후 처리

트랜잭션 제거

블록이 커밋되면 포함된 트랜잭션들이 메모리풀에서 제거됩니다:

```
func (mem *CListMempool) Update(height int64, txs types.Txs, txResults []*abci.
// 1. 높이 업데이트
mem.height.Store(height)
mem.notifiedTxsAvailable.Store(false)
// 2. 커밋된 트랜잭션 처리
for i, tx := range txs {
if txResults[i].Code == abci.CodeTypeOK {
// 성공한 트랜잭션은 캐시에 추가
mem.addToCache(tx)
} else {
// 실패한 트랜잭션은 캐시에서 제거
```

```
mem.tryRemoveFromCache(tx)
}
// 메모리풀에서 제거
if err := mem.RemoveTxByKey(tx.Key()); err != nil {
mem.logger.Debug("Committed transaction not in local mempool", "tx", tx.Hash())
}
}
// 3. 남은 트랜잭션 재검증
if mem.config.Recheck {
mem.recheckTxs()
}
return nil
}
```

재검증 (Recheck) 과정

새 블록 커밋 후 남은 트랜잭션들의 유효성을 재확인합니다:

```
func (mem *CListMempool) recheckTxs() {
if mem.Size() <= 0 {
return
}
mem.recheck.init()
// 모든 트랜잭션에 대해 재검증 요청
```

```
iter := NewNonBlockingIterator(mem)
for {
memTx := iter.Next()
if  memTx == nil {
break
}
mem.recheck.numPendingTxs.Add(1)
// 재검증 요청 전송
resReq, err := mem.proxyAppConn.CheckTxAsync(context.TODO(), &abci.CheckTxReques
Tx: memTx.Tx(),
Type: abci.CHECK_TX_TYPE_RECHECK,
})
if  err != nil {
panic(fmt.Errorf("recheck request failed: %w", err))
}
resReq.SetCallback(mem.handleRecheckTxResponse(memTx.Tx()))
}
// 응답 대기
mem.proxyAppConn.Flush(context.TODO())
// 타임아웃 또는 완료 대기
select {
case <-time.After(mem.config.RecheckTimeout):
mem.logger.Error("Timed out waiting for recheck responses")
```

```
case <-mem.recheck.doneRechecking():

}

}
```

성능 최적화 요소

1. 비동기 처리

- CheckTx 요청은 비동기로 처리되어 블로킹 방지
- 콜백 기반 응답 처리로 높은 처리량 달성

2. 효율적인 데이터 구조

- 해시맵 기반 O(1) 트랜잭션 검색
- 연결 리스트 기반 O(1) 추가/제거
- LRU 캐시로 중복 처리 최적화

3. 동시성 제어

- 읽기-쓰기 뮤텍스로 동시 읽기 허용
- 세밀한 락킹으로 경합 최소화

4. 메모리 관리

- 레인별 용량 분할로 공정성 보장
- 적응적 캐시 크기 조정

참고 자료

- 메모리풀 개요
- ABCI 인터페이스
- P2P 메시지 전파